Tab 1

🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

A Cisco Networking Academy–style lab that blends classical networking with application-layer engineering. You will configure inter-VLAN routing, deploy a MIME-aware socket server, capture packets, and analyze how meaning flows through the stack.

ปภาวรินทร์ นาเมืองรักษ์	673380275-5	Part 4 — Wireshark Capture Walkthrough on PC1 and PC1, Part 3__On Client (PC1)
ปรนันท์ บุสดีวงค์	673380276-3	Part5,Part3(On Server (PC2),Assessment Questions(1,2,3),Reflection Exercise
วิภาวี ฤทธิหาญ	673380514-3	Part 1, Part 2, ตอบ Assessment Questions(4,5)

📘 Lab Overview

Scenario

A client in one VLAN transfers files to a server in another VLAN using a custom MIME-based protocol (JSON header + binary payload). You will verify connectivity, observe encapsulation, and analyze packets using Wireshark.

This lab reinforces how Layer 2 segmentation, Layer 3 routing, TCP reliability, and application metadata interact.

🎯 Objectives

After completing this lab, you will be able to:

- Configure Router-on-a-Stick inter-VLAN routing
- Configure VLANs and trunking on a switch
- Deploy and run a MIME socket server and client

- Capture traffic using Wireshark
- Identify ARP, VLAN tags, TCP handshake, and application data
- Decode JSON headers in packet payload
- Troubleshoot network and application failures
- Document protocol behavior across OSI layers

🧰 Required Resources

- 1 Router (e.g., Cisco Systems ISR/virtual router)
- 1 Switch (Layer 2)
- 2 PCs or VMs
- Ethernet cables or virtual links
- Python 3 installed
- MIME file transfer project
- Wireshark installed (from Wireshark Foundation)

🗺️ Topology

```
None
PC1 (VLAN 10) ----\
                    \ access
                     \
                     [S1]---- trunk ----[R1]
                     /
                    / access
PC2 Server (VLAN 20)
```

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

📋 Addressing Scheme

🧠 Background Theory

Router-on-a-Stick

A single physical interface carries multiple VLANs via 802.1Q tagging. Subinterfaces perform routing between broadcast domains.

MIME Metadata

MIME labels content type (e.g., image/png). Your protocol wraps files with JSON describing:

- MIME type
- Size

Why This Matters

Real systems — APIs, storage services, message brokers — depend on this exact pattern: structured metadata + binary payload over reliable transport.

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```
None
enable
configure terminal
hostname R1

interface g0/0
no shutdown

interface g0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0
```

```
end
write memory
```

Screenshots of configurations :

```
R1>enable
R1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
R1(config)#hostname R1
R1(config)#interface g0/0/0
R1(config-if)#no shutdown
R1(config-if)#interface g0/0/0.10
R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#interface g0/0/0.20
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0/0.20, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0.20,
changed state to up

R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
R1#
```

Step 2 — Switch Configuration

```
None
enable
configure terminal
hostname S1

vlan 10
name CLIENTS

vlan 20
name SERVERS

interface g0/1
switchport mode trunk

interface fa0/1
switchport mode access
switchport access vlan 10

interface fa0/2
switchport mode access
switchport access vlan 20

end
write memory
```

Screenshots of configurations :

```
S1#enable
S1#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
S1(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#interface fa0/1
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#
S1(config-if)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switchport access vlan 20
S1(config-if)#end
S1#write memory
Building configuration...
[OK]
S1#
%SYS-5-CONFIG_I: Configured from console by console
```

Step 3 — Configure Hosts

Set static IP addresses per table.

🔍 Part 2 — Verify Connectivity

From PC1:

```
None
ping 192.168.20.20
```

Verify:

- Successful replies
- ARP entries

```
Cisco Packet Tracer PC Command Line 1.0
C:\>show ip interface brief
Invalid Command.

C:\>ping 192.168.20.20

Pinging 192.168.20.20 with 32 bytes of data:

Reply from 192.168.20.20: bytes=32 time=2ms TTL=128
Reply from 192.168.20.20: bytes=32 time=4ms TTL=128
Reply from 192.168.20.20: bytes=32 time=3ms TTL=128
Reply from 192.168.20.20: bytes=32 time=4ms TTL=128

Ping statistics for 192.168.20.20:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 2ms, Maximum = 4ms, Average = 3ms

C:\>
```

Router checks:

```
None
show ip route
show ip interface brief
```

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.0.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.0.0/24 is directly connected, GigabitEthernet0/0/0
L       192.168.0.1/32 is directly connected, GigabitEthernet0/0/0
     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0/0.10
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0/0.20

R1#show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0/0   192.168.0.1     YES manual up                    up
GigabitEthernet0/0/0.10192.168.10.1    YES manual up                    up
GigabitEthernet0/0/0.20192.168.20.1    YES manual up                    up
GigabitEthernet0/0/1   unassigned      YES unset  administratively down down
GigabitEthernet0/0/2   unassigned      YES unset  administratively down down
Serial0/1/0            unassigned      YES manual down                  down
Serial0/1/1            unassigned      YES unset  administratively down down
Vlan1                  unassigned      YES unset  administratively down down
R1#
```

Copy Paste

Switch checks:

```
None
show interfaces trunk
show mac address-table
```

```
S1>show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,2,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,2,10,20

S1>show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    00d0.58b5.0401    DYNAMIC     Gig0/1
  10    00d0.58b5.0401    DYNAMIC     Gig0/1
  20    0000.0ce1.b994    DYNAMIC     Fa0/2
  20    00d0.58b5.0401    DYNAMIC     Gig0/1
S1>
```

🧪 Part 3 — Deploy MIME Server

On Server (PC2)

```
None
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

```
PS C:\Users\LOQ\automation> cd week01-mime-typing
PS C:\Users\LOQ\automation\week01-mime-typing> python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
[18:55:56] INFO: Starting server with host=0.0.0.0, port=65432, storage=storage
[18:55:56] INFO: Server listening on 0.0.0.0:65432
```

ภาพรอ client

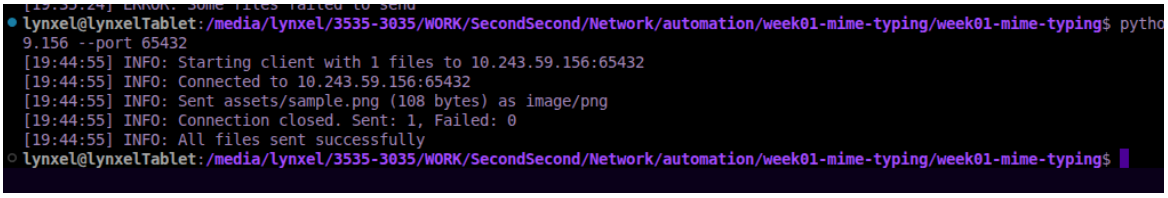
```
PS D:\Automation\automation\week01-mime-typing> python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
[19:44:51] INFO: Starting server with host=0.0.0.0, port=65432, storage=storage
[19:44:51] INFO: Server listening on 0.0.0.0:65432
[19:44:55] INFO: Connection from ('10.243.59.198', 52714)
[19:44:55] INFO: Receiving image/png (108 bytes)
[19:44:55] INFO: Saved to storage\received_4ce9.png
[19:44:55] INFO: Client ('10.243.59.198', 52714) closed connection
[19:44:55] INFO: Connection with ('10.243.59.198', 52714) closed
```

ภาพหลังจาก rece

On Client (PC1)

```
None
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port 65432

python client/main_enhanced.py assets/sample.png --host 192.168.1.53 --port 65432
```



Confirm file saved.

📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

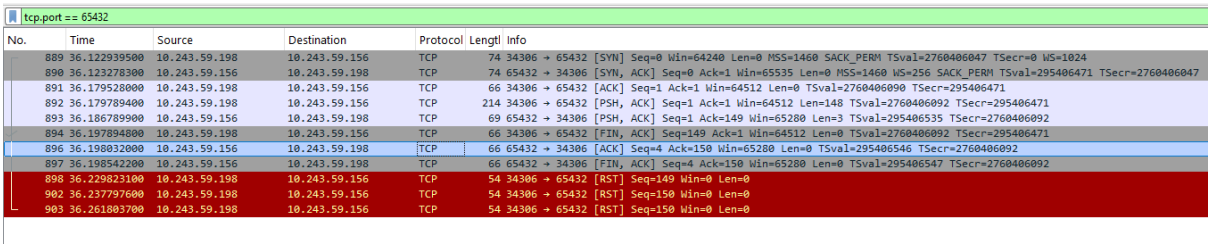
Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

Apply capture filter:

```
None
tcp port 65432
tcp.port==65432
```



[SYN] (No. 889): จาก Client (`.198`) ส่งไป Server (`.156`) เพื่อขอเริ่มคุย
[SYN, ACK] (No. 890): Server ตอบรับการเชื่อมต่อกลับมา
[ACK] (No. 891): Client ยืนยันการเชื่อมต่อขั้นสุดท้าย

Step 2 — Initiate Transfer

Send file again.
{"mime_type": "image/png", "size": 108}
.PNG
.
xxxx

arp

No.	Time	Source	Destination	Protocol	Lengtl	Info
856	20.032002700	7c:3d:09:62:ca:ae	46:60:ce:c6:3b:65	ARP	42	10.243.59.156 is at 7c:3d:09:62:ca:ae
959	26.951933800	7c:3d:09:62:ca:ae	Intel_f4:cd:0b	ARP	42	10.243.59.156 is at 7c:3d:09:62:ca:ae
855	20.031900300	46:60:ce:c6:3b:65	7c:3d:09:62:ca:ae	ARP	42	Who has 10.243.59.156? Tell 10.243.59.166
958	26.951849900	Intel_f4:cd:0b	7c:3d:09:62:ca:ae	ARP	42	Who has 10.243.59.156? Tell 10.243.59.198

```
> Frame 958: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{AF256677-F807-43DB-A34B-62D821
v Ethernet II, Src: Intel_f4:cd:0b (00:24:d6:f4:cd:0b), Dst: 7c:3d:09:62:ca:ae (7c:3d:09:62:ca:ae)
  v Destination: 7c:3d:09:62:ca:ae (7c:3d:09:62:ca:ae)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  v Source: Intel_f4:cd:0b (00:24:d6:f4:cd:0b)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: ARP (0x0806)
    [Stream index: 1]
> Address Resolution Protocol (request)
```

Confirm MAC resolution.

Source MAC: `7c:3d:09:62:ca:ae` (เครื่องส่ง)

Destination MAC: `Intel_f4:cd:0b` (เครื่องรับ)

Protocol: TCP

Port: 34306 (Client) -> 65432 (Server)

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Source	Destination	Protocol	Lengtl	Info
10.243.59.198	10.243.59.156	TCP	74	42844 → 65432 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2762486264 TSecr=0 WS=1024
10.243.59.156	10.243.59.198	TCP	74	65432 → 42844 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=297486624 TSecr=2762486264
10.243.59.198	10.243.59.156	TCP	66	42844 → 65432 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2762486305 TSecr=297486624

SYN (No. 889): Client ส่งคำขอเชื่อมต่อ เริ่มต้นด้วย Sequence Number = 0

SYN, ACK (No. 890): Server ตอบกลับ ยืนยันว่าพร้อมรับข้อมูล

ACK (No. 891): Client ยืนยันกลับไปว่า "รับทราบ" ตอนนี้ท่อข้อมูลเปิดแล้ว

Check sequence numbers.

Source	Destination	Protocol	Lengtl	Info
10.243.59.198	10.243.59.156	TCP	74	59482 → 65432 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2763391727 TSecr=0 WS=1024
10.243.59.156	10.243.59.198	TCP	74	65432 → 59482 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM TSval=298392051 TSecr=2763391727
10.243.59.198	10.243.59.156	TCP	66	59482 → 65432 [ACK] Seq=1 Ack=1 Win=64512 Len=0 TSval=2763391740 TSecr=298392051
10.243.59.198	10.243.59.156	TCP	214	59482 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=64512 Len=148 TSval=2763391748 TSecr=298392051

SYN seq=0 : 1887240516

SYM-ACK seq=0 : 418458083

ACK seq=1 : 1887240517

C — Data Transfer

Follow stream:

10.243.59.198	10.243.59.156	TCP	214	59482 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=64512 Len=148 TSval=2763391748 TSecr=298392051
10.243.59.156	10.243.59.198	TCP	69	65432 → 59482 [PSH, ACK] Seq=1 Ack=149 Win=65280 Len=3 TSval=298392081 TSecr=2763391748

Right click → Follow → TCP Stream.

You should see JSON header similar to:

```
None
{"mime_type":"image/png","size":12345}
```

We found this: {"mime_type": "image/png", "size": 108}
.PNG
.
xxxx

🔬 Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior
TCP	Flags	Reliability
Payload	JSON	MIME metadata

🛠️ Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout

Ping failure : เมื่อทดสอบ Ping จะขึ้น Request timed out เพราะไม่มีประตู สำหรับส่งข้อมูลข้าม VLAN

TCP timeout : โค้ด Python ฝั่ง Client จะพยายามเชื่อมต่อนานผิดปกติ และสุดท้ายจะแจ้งเตือนว่าการเชื่อมต่อล้มเหลวเพราะ Packet ไปไม่ถึงปลายทาง

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

SYN retransmissions : ใน Wireshark จะเห็น Packet ประเภท ของ SYN บ่อยครั้ง เนื่องจากเครื่อง Client พยายามส่งคำขอเชื่อมต่อไปแล้ว แต่ไม่ได้รับการตอบกลับเพราะไฟร์วอลล์ดรอปข้อมูลทิ้ง ทำให้ Client ต้องส่งซ้ำเรื่อยๆ

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete

ARP incomplete: เมื่อดูในตาราง ARP หรือดูใน Wireshark จะพบว่าเครื่องส่งพยายามถามหา MAC Address ของ Gateway ที่ไม่มีอยู่จริง แต่ไม่มีการตอบกลับ ทำให้การสร้างเฟรมข้อมูลไม่สมบูรณ์และส่งข้อมูลออกไปไม่ได้

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions
- Window adjustment

TCP retransmissions : เมื่อมีข้อมูลบางส่วนหายไประหว่างทาง TCP จะตรวจพบและทำการส่งเฉพาะข้อมูลส่วนที่หายไปนั้นใหม่

Window adjustment : จะเห็นค่า **Window Size** ใน TCP Header มีการเปลี่ยนแปลงลดลง เพื่อควบคุมการไหลของข้อมูลไม่ให้ส่งเร็วเกินไปจนระบบรับไม่ไหวหลังจากตรวจพบการสูญหาย

📝 Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence

- Encapsulation explanation
- Troubleshooting notes

🎓 Assessment Questions

1. Why does TCP retransmit lost segments?
   ตอบ : เพราะ TCP เป็นโปรโตคอลที่เน้นความน่าเชื่อถือ โดยมีกลไกตรวจสอบว่าข้อมูลส่งไปถึงปลายทางครบถ้วนหรือไม่ หากเครื่องส่งไม่ได้รับสัญญาณยืนยัน ภายในเวลาที่กำหนด หรือได้รับสัญญาณว่าข้อมูลหายไป TCP จะทำการส่งข้อมูลชุดนั้นซ้ำ
2. How does VLAN tagging affect broadcast domains?
   ตอบ : การทำ VLAN tagging ช่วยในการแบ่ง หรือจำกัดขอบเขตของ Broadcast Domain โดยปกติถ้าไม่มี VLAN ทุกเครื่องใน Switch จะได้รับข้อมูล Broadcast ถึงกันหมด แต่เมื่อมี VLAN tagging ข้อมูลจะถูกส่งต่อไปเฉพาะในกลุ่ม VLAN ID เดียวกันเท่านั้น แม้จะอยู่บน Switch หรือสายสัญญาณเส้นเดียวกันก็ตาม ซึ่งช่วยลดความคับคั่งของเครือข่ายและเพิ่มความปลอดภัย
3. What role does MIME play compared to TCP headers?
   ตอบ : TCP รับผิดชอบให้ข้อมูลไปถึงแต่ MIME รับผิดชอบให้เครื่องปลายทางเข้าใจว่าข้อมูลที่ได้รับคืออะไร
4. Where does routing decision occur?
   ตอบ : การตัดสินใจเลือกเส้นทางเกิดขึ้นที่ Layer 3 หรือ Network Layer โดย Router จะตรวจสอบ Destination IP Address ของข้อมูลที่เข้ามา แล้วเปรียบเทียบกับ Routing Table เพื่อเลือกเส้นทางหรือ Interface ที่เหมาะสมที่สุดในการส่งต่อข้อมูลไปยังปลายทาง
5. Why does ARP precede TCP communication?
   ตอบ : ARP ต้องเกิดก่อนการสื่อสารแบบ TCP เพราะก่อนที่อุปกรณ์จะส่งข้อมูล TCP ได้ จำเป็นต้องทราบ MAC Address ของปลายทางก่อน โดย ARP ทำหน้าที่แปลง IP Address ให้เป็น MAC Address เพื่อให้สามารถส่งข้อมูลผ่านเครือข่ายในระดับ Layer 2 ได้อย่างถูกต้อง

📊 Reflection Exercise

Explain:

How does application meaning survive multiple encapsulation layers?

ตอบ ความหมายของข้อมูลสามารถคงอยู่และถูกส่งต่อไปถึงปลายทางได้ด้วยหลักการ Encapsulation และ Decapsulation ตามลำดับชั้นของ OSI Model ดังนี้

1. การรักษาลำดับชั้น: ข้อมูลเริ่มต้นจากความตั้งใจของผู้ใช้ในระดับ Application Layer โดยมีการใช้ MIME Metadata (เช่น JSON header ใน Lab นี้) เพื่อกำกับว่าข้อมูลคืออะไร

2. การหุ้มข้อมูล: เมื่อข้อมูลถูกส่งลงไปในเลเยอร์ที่ต่ำกว่า (เช่น TCP, IP, Ethernet) ข้อมูลในระดับบนจะถูกมองว่าเป็นเพียงPayload หรือเนื้อหาดิบที่เลเยอร์ล่างไม่จำเป็นต้องเข้าใจความ

หมาย แต่จะทำหน้าที่หุ้มข้อมูลนั้นด้วย Header ของตนเอง เพื่อจัดการเรื่องความน่าเชื่อถือ และที่อยู่

3. การถอดรหัสที่ปลายทาง: เมื่อสัญญาณไปถึงปลายทาง แต่ละเลเยอร์จะถอด Header ของตัวเองออกและส่ง Payload ต่อขึ้นไปด้านบน จนกระทั่งถึงเลเยอร์บนสุด ข้อมูล JSON และ Binary ที่ถูกหุ้มไว้ก็จะถูกเปิดออกอีกครั้ง

4. การตีความความหมาย: แม้ข้อมูลจะถูกเปลี่ยนเป็นสัญญาณไฟฟ้าหรือเฟรมข้อมูลในระหว่างทาง แต่เมื่อถึงปลายทาง แอปพลิเคชันจะใช้ MIME Metadata ที่ส่งมาด้วยในการตีความว่าไบต์เหล่านั้นคือไฟล์รูปภาพประเภทใด ทำให้ความหมายของไฟล์กลับมาสมบูรณ์เหมือนต้นทาง

⚠️ Common Pitfalls

- Server bound to localhost only
- Missing trunk
- Wrong VLAN
- OS firewall enabled
- Capture on wrong interface

🚀 Extension Activities

- Add TLS encryption
- Measure throughput vs latency
- Capture checksum errors
- Add checksum field to protocol
- Containerize server
- Compare with HTTP transfer

🧬 Instructor Notes — What This Lab Really Teaches

This is not just networking.

It demonstrates:

- Protocol layering philosophy
- Reliability engineering

- Metadata semantics
- Distributed system fundamentals

The student watches a file become:

intention → bytes → frames → signals → bytes → meaning

That transformation is the essence of computer networks.

TCP ping

Here's how you can modify the script to accept an IP address and port number as command-line arguments using sys.argv:

```python
import sys
from tcppinglib import tcpping
import asyncio

def main():
    # Check if correct number of arguments is provided
    if len(sys.argv) != 3:
        print("Usage: python script.py <IP_ADDRESS> <PORT>")
        sys.exit(1)

    ip_address = sys.argv[1]
    try:
        port = int(sys.argv[2])
    except ValueError:
        print("Error: Port must be an integer.")
        sys.exit(1)

    # Perform TCP ping
    host_stats = tcpping(ip_address, port=port, count=4, interval=1)

    print(f"Host: {host_stats.ip_address}")
    print(f"Is alive: {host_stats.is_alive}")
    print(f"Average RTT: {host_stats.avg_rtt:.2f} ms")
    print(f"Packet loss: {host_stats.packet_loss}%")

if __name__ == "__main__":
    main()
```

Explanation:

· The script uses sys.argv to read command-line arguments.
· It expects two arguments: the IP address (or hostname) and the port number.
· Basic error handling ensures both arguments are provided and the port is a valid integer.
· The tcpping function is called with the provided IP and port, keeping the same count and interval for demonstration.

You can run it like:

```bash
python script.py 192.168.1.1 80
```

If you also want to include the asynchronous version, you can adapt it similarly, but the synchronous example above covers the core requirement.

Role-based procedures

👥 Role-Based Lab Procedure: MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

🎭 Team Roles

Role Primary Responsibilities Suggested Tools
Network Infrastructure Engineer Configures router and switch, sets up VLANs and trunking, assigns IP addresses to router subinterfaces, verifies Layer 2/3 connectivity, troubleshoots network issues. Console access (PuTTY, SecureCRT), Cisco IOS CLI, ping, traceroute, show commands, possibly Packet Tracer for simulation.
Client/Server Engineer Deploys and runs the MIME server and client scripts on the two PCs, initiates file transfers, captures traffic with Wireshark, observes application behavior. Python 3, terminal/command prompt, Wireshark, the provided server/main_enhanced.py and client/main_enhanced.py scripts, a sample file (e.g., sample.png).
Knowledge Manager / Supervisor Oversees the lab workflow, documents configurations and results, leads analysis of packet captures, verifies understanding of concepts, coordinates troubleshooting, compiles the final report. Note-taking app (OneNote, Google Docs), Wireshark (for detailed analysis), lab manual, diagramming tool (draw.io), camera/screenshot tool.

📋 Step-by-Step Procedure

All three team members should be present and collaborate. The Knowledge Manager ensures every step is understood and documented.

Part 1 — Configure Network Devices

Network Infrastructure Engineer

· Access the router (R1) console and configure:
  · Hostname, interface G0/0, subinterfaces G0/0.10 and G0/0.20 with 802.1Q encapsulation and IP addresses per the addressing table.
  · Issue no shutdown on the physical interface.
  · Save configuration with write memory.
· Access the switch (S1) console and configure:
  · VLANs 10 and 20.
  · Interface G0/1 as trunk.
  · Interfaces Fa0/1 (access VLAN 10) and Fa0/2 (access VLAN 20).
  · Save configuration.
· Verify configurations with show ip interface brief, show interfaces trunk, show vlan brief.

Client/Server Engineer

· Not actively involved in this part, but observe the configuration process and take notes of the commands used (with help from the Knowledge Manager).

Knowledge Manager / Supervisor

· Document all configuration commands and their expected effects.
· Take screenshots of the running configurations.
· Ensure that the physical cabling matches the topology (PC1 connected to Fa0/1, PC2 to Fa0/2, router G0/0 to switch G0/1).
· Briefly explain the purpose of trunking and subinterfaces to the team.

Part 2 — Verify Connectivity

Network Infrastructure Engineer

· From the router, ping both PC IP addresses to confirm router-to-host connectivity:
```
ping 192.168.10.10
ping 192.168.20.20
```
· Use show ip route to confirm routes to both subnets are present (directly connected).

Client/Server Engineer

· On PC1, set the static IP address 192.168.10.10/24 with gateway 192.168.10.1.
· On PC2, set the static IP address 192.168.20.20/24 with gateway 192.168.20.1.
· From PC1, ping PC2:
```
ping 192.168.20.20
```
· If pings fail, work with the Network Infrastructure Engineer to troubleshoot (check VLAN assignment, trunk, IP settings).

Knowledge Manager / Supervisor

· Record successful ping results (screenshot).
· Verify that ARP entries are populated on both PCs (arp -a).
· Ensure the team understands why ping works across VLANs (router-on-a-stick provides inter-VLAN routing).

Part 3 — Deploy MIME Server and Transfer a File

Client/Server Engineer

· On PC2 (server) , open a terminal and start the server:
```bash
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```
· On PC1 (client) , open another terminal and send a file:
```bash
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port 65432
```
· Confirm that the file is saved on the server side (e.g., check the destination folder).
· If the transfer fails, verify that the server is listening on 0.0.0.0 (not localhost) and that no firewall is blocking port 65432.

Network Infrastructure Engineer

· Monitor network statistics during transfer (optional, e.g., show interface counters).
· Assist in troubleshooting connectivity if needed.

Knowledge Manager / Supervisor

· Document the exact commands used and their output.
· Ensure the team understands the MIME header concept (JSON metadata + binary payload).
· Take a screenshot of the server log showing the received file information.

Part 4 — Wireshark Capture and Analysis

Client/Server Engineer (both PC1 and PC2)

· On both PCs, launch Wireshark and start capturing on the network interface connected to the switch.
· Apply a capture filter to isolate the traffic:
```
tcp port 65432
```
· On PC1, re-run the file transfer command (the client).
· Stop the capture on both machines after the transfer completes.
· Save the capture files (e.g., client_capture.pcapng, server_capture.pcapng).

Knowledge Manager / Supervisor

· Lead the analysis of the captured packets.
  · On PC1's capture, locate:
    · ARP request/reply for the server's IP (if not already cached).
    · TCP three-way handshake (SYN, SYN-ACK, ACK).

  · Data packets carrying the JSON header (right-click → Follow → TCP Stream to see the full conversation).
  · The binary file data.
 · On PC2's capture, compare the same packets from the server's perspective.
· If capturing on the trunk link (optional), look for 802.1Q tags with VLAN IDs 10 and 20.
· For each layer (Ethernet, 802.1Q, IP, TCP, Application), note the fields listed in the dissection guide.
· Take screenshots of:
 · ARP packet details.
 · TCP handshake packets.
 · The JSON header inside the TCP stream.
 · A packet containing binary payload.
 · (Optional) VLAN tag information.

Network Infrastructure Engineer

· Assist in interpreting VLAN tags if trunk capture was performed.
· Explain how the router decides to forward packets based on destination IP and VLAN.

Part 5 — Troubleshooting Scenarios

The team works together to simulate failures and observe the effects. The Knowledge Manager predicts the expected behavior, then the group implements the fault and observes.

Scenario Action by … Expected Observation Documented Result
Broken Routing Network Infrastructure Engineer shuts down subinterface G0/0.10 on the router. Ping from PC1 to PC2 fails; TCP SYN from client times out (retransmissions). Screenshot of failed ping and Wireshark showing SYN retransmits.
Firewall Block Client/Server Engineer (on server) adds a firewall rule to block port 65432. Client's TCP SYN is unanswered; Wireshark shows repeated SYN attempts. Screenshot of firewall rule and capture showing no SYN-ACK.
Wrong Gateway Client/Server Engineer changes PC1's gateway to an incorrect IP. Ping fails; ARP requests for the gateway remain unanswered ("ARP incomplete"). Screenshot of arp -a showing incomplete entry.
Packet Loss Network Infrastructure Engineer uses a traffic control tool (e.g., tc on Linux) to drop 10% of packets. TCP retransmissions and possible window size adjustment; file still transfers reliably. Wireshark capture showing duplicate ACKs and retransmissions.

Knowledge Manager / Supervisor records each observation and leads a discussion on why TCP behaves the way it does.

📝 Lab Report Compilation

The Knowledge Manager is responsible for assembling the final lab report, incorporating:

· Screenshots from all parts (configs, ping results, Wireshark captures, JSON header).
· Explanations of each observation, linking to OSI layers.
· Answers to the assessment questions.
· A reflection on how application-layer meaning survives encapsulation.

The Network Infrastructure Engineer and Client/Server Engineer provide their notes, captures, and any troubleshooting logs.

✅ Collaboration Checkpoints

At the end of each major part, the team should pause and:

· Review what was accomplished.
· Ensure all documentation (screenshots, notes) is saved.
· Discuss any unexpected issues and how they were resolved.

This role-based approach ensures every student actively contributes while learning both the technical and analytical aspects of the lab.